

NutriCare Connect

Web-based Diet Planning & Health Check-up System

Software Engineering Academic Pack

SE2030 - Software Engineering
Year 2, Semester 1 - 2026
Group 2026-Y2-S1-MLB-B5G1-01

Academic demonstration only. No real patient, clinical, or payment data.

Software Requirements Specification

Purpose

NutriCare Connect centralizes patient registration, scheduling, health checks, diet plans, communication, billing, feedback and management reporting for a private wellness centre. This implementation is an academic demonstration and must never be used for clinical decisions or real payment processing.

Users and goals

- **Patient:** manage profile and appointments; view personal results, plans, progress, invoices and notifications; message their assigned dietitian; submit feedback.
- **Dietitian:** view assigned patients, respond to alerts, publish diet plans and monitor progress.
- **Doctor / medical coordinator:** record check-ups and recommendations and schedule follow-ups.
- **Reception staff:** register patients and manage schedules and appointment changes.
- **System administrator:** manage accounts, roles, locks and audit activity.
- **Operations manager:** review visits, retention, workload, satisfaction and complaints.
- **Finance executive:** review invoices, demo payments, refunds and financial summaries.
- **Patient relations officer:** monitor questions, service issues and complaint follow-up.

Functional requirements

ID	Requirement	Acceptance summary
FR-01	Register and authenticate users	Email is unique, password is BCrypt hashed, JWT is issued after valid login.
FR-02	Enforce roles and permissions	Protected API and UI actions are unavailable to unauthorized roles.
FR-03	Lock unsafe login attempts	Five consecutive failures lock the account; an administrator can unlock it.
FR-04	Manage patient records	Authorized users can create and view profile and medical-history information.
FR-05	Manage practitioner schedules	Available slots are returned by date and practitioner.
FR-06	Book without conflicts	A transactional lock and unique database constraint prevent double booking.
FR-07	Reschedule or cancel	Status history is retained and cancelled slots become available.
FR-08	Generate invoices and demo payments	Payment status and audit metadata are saved without sensitive card data.
FR-09	Record health checks	Weight, BMI, blood pressure, blood sugar, temperature and notes are stored.
FR-10	Trigger health alerts	Configured demo thresholds create prioritized dietitian alerts.
FR-11	Create and publish diet plans	A plan contains targets, exclusions and a meal schedule.

ID	Requirement	Acceptance summary
FR-12	Track progress	Patients log meals, water, weight and BMI; charts show trends.
FR-13	Securely message care staff	Messages are restricted to authorized patient-care relationships.
FR-14	Send reminders	In-app reminders work; email/SMS attempts are simulated and retried after failure.
FR-15	Collect feedback	Ratings from 1-5 and optional comments are stored after consultation.
FR-16	Escalate complaints	A rating of 2 or below creates an urgent complaint ticket.
FR-17	Produce reports	Date-filtered operational, clinical, satisfaction and financial summaries are available.
FR-18	Record audit events	Security-sensitive, clinical, financial and administrative mutations identify actor and time.

Non-functional requirements

- Responsive operation from 320 px mobile screens through desktop dashboards.
- Keyboard-accessible controls, visible focus states, semantic labels and readable contrast.
- API validation returns stable error codes and field-level messages.
- Database transactions protect booking and payment state transitions.
- No raw password, full card number, CVV or real patient data is logged or seeded.
- All timestamps are stored consistently and presented in Asia/Colombo time for the demo.
- Module boundaries allow each member folder to be reviewed or uploaded independently.

Out of scope

Production clinical guidance, regulatory certification, real payment gateways, real SMS/email delivery, insurance processing, prescription management and real-world emergency response are explicitly excluded.

Architecture and Module Boundaries

Context

```
flowchart LR
    U[Patients and staff] --> UI[React + Vite]
    UI --> REST[REST /api/v1 + JWT]
    REST --> API[Spring Boot application]
    API --> A[User access]
    API --> B[Appointments and billing]
    API --> C[Health checks]
    API --> D[Diet and progress]
    API --> E[Messaging and reminders]
    API --> F[Feedback and analytics]
    A & B & C & D & E & F --> DB[(MySQL nutricare)]
    E -. simulated .-> X[Email / SMS delivery log]
    B -. simulated .-> P[Payment status recorder]
```

Composition

The root Maven project builds six feature jars and the integration/backend executable. Component scanning discovers every controller, service, repository and entity under `lk.sliit.nutricare`. The Vite application imports one React feature package from each student folder and supplies the common shell, navigation and design tokens.

Cross-module records use UUID values rather than Java entity relationships. This keeps compile-time ownership clear while allowing database indexes and API contracts to join data. Feature modules do not import another member's persistence classes; integration occurs through IDs, stable REST operations, and the shared SQL contract. Authentication is supplied by Module 01 through Spring Security. Changes that affect another module require an OpenAPI change and team review.

Data flow

1. Login issues an eight-hour HMAC-signed JWT containing user ID, email and role.
2. The browser sends the token as `Authorization: Bearer ...`.
3. Controllers validate input; services enforce state transitions inside transactions.
4. JPA repositories persist records to the shared database created by Flyway migrations.
5. API responses exclude password hashes and prohibited payment credentials.

Standard API behavior

- Prefix: `/api/v1`
- Content type: `application/json`
- IDs: UUID strings
- Validation: HTTP 400 with `code=VALIDATION_ERROR` and `fields`
- State conflict: HTTP 409 with `code=CONFLICT`
- Missing authentication: HTTP 401; insufficient permission: HTTP 403
- Timestamps: ISO-8601

Database Design and Data Dictionary

Entity relationship overview

```
erDiagram
    USER_ACCOUNTS ||--o{ AUDIT_EVENTS : performs
    USER_ACCOUNTS ||--o{ APPOINTMENTS : patient_or_practitioner
    AVAILABILITY_SLOTS ||--o{ APPOINTMENTS : reserves
    APPOINTMENTS ||--|| INVOICES : generates
    INVOICES ||--o{ PAYMENTS : receives
    USER_ACCOUNTS ||--o{ HEALTH_CHECKS : patient
    HEALTH_CHECKS ||--o{ HEALTH_ALERTS : triggers
    USER_ACCOUNTS ||--o{ DIET_PLANS : patient
    DIET_PLANS ||--o{ PROGRESS_LOGS : tracks
    USER_ACCOUNTS ||--o{ SECURE_MESSAGES : sends_or_receives
    USER_ACCOUNTS ||--o{ NOTIFICATIONS : receives
    APPOINTMENTS ||--o{ FEEDBACK : reviewed_by
    FEEDBACK ||--o{ COMPLAINTS : escalates
```

Table ownership

Migration	Tables	Owner
V1	user_accounts, audit_events	Module 01
V2	availability_slots, appointments, invoices, payments	Module 02
V3	health_checks, health_alerts	Module 03
V4	diet_plans, progress_logs	Module 04
V5	secure_messages, notifications	Module 05
V6	feedback, complaints	Module 06
V7	Safe synthetic demo users and slots	Integration

UUIDs are stored as `BINARY(16)`. Monetary values use `DECIMAL(12, 2)`. Clinical measurements use fixed-point decimals. Long user-entered text is length limited. Every history/report path has a patient, time or status index appropriate to its query.

Migration policy

- Never edit a migration already shared with another member; add the next version.
- Migrations must be backward-safe for existing demo data.
- Schema names use lower-case snake case; Java fields use camel case.
- Sample records must be fictional and visibly demo-only.
- Before merging, start a fresh MySQL container and run all migrations from V1.

Security and Privacy Guide

Controls implemented

- BCrypt password hashing and constant-time JWT signature comparison.
- Five-attempt account lock with administrator unlock operation.
- Stateless bearer authentication and server-side role authorities.
- Request validation, bounded text fields, parameterized repository queries and transactional writes.
- Booking row locks, optimistic versioning and a database uniqueness constraint.
- Audit records for registration and login, with the extension point documented for all mutations.
- CORS limited to the configured frontend origin.

Role visibility

Data/action	Patient	Care staff	Reception	Admin	Manager	Finance
Own profile/results/plan	Own only	Assigned patients	Minimum registration data	Account metadata	Aggregate only	No clinical data
Scheduling	Own	Own schedule	All schedules	Support only	Aggregate only	Invoice link only
Clinical entry	No	Doctor/coordinator	No	No content access	Aggregate only	No
Diet plan editing	No	Dietitian	No	No	Aggregate only	No
Payments	Own history	No	Record cash	Audit metadata	Aggregate	Full demo ledger
Complaints	Own	Assigned response	Refer	Manage	Manage	No

Prohibited data

Never enter or persist real names tied to health data, national identifiers, real payment-card numbers, CVV, online-banking credentials, insurance records or production medical results. The payment request accepts only amount, method and status. Medical thresholds are marked non-diagnostic.

Before any public demonstration

Replace the JWT secret and default passwords, recreate the database from safe seed data, verify logs contain no confidential values, and keep the API/database on a private development network.

Verification and Acceptance Test Plan

Levels

- Unit: domain state changes such as login locking, slot holding and feedback rating handling.
- Repository: Flyway migration, uniqueness, locking and indexed lookup behavior against MySQL.
- API: validation, authentication, role authorization and response contracts.
- UI: form behavior, navigation, responsive layout and accessible labels.
- End to end: the complete patient journey with synthetic records.

Acceptance matrix

Test	Scenario	Expected result
AT-01	Register valid patient	Account is created with hashed password and PATIENT role.
AT-02	Submit invalid email or short password	HTTP 400 identifies invalid fields.
AT-03	Fail login five times	Account becomes locked; later valid password is rejected.
AT-04	Access clinical endpoint as reception	HTTP 403 and no clinical record is returned.
AT-05	Hold the same slot concurrently	One request succeeds; the other receives a conflict/rejection.
AT-06	Leave held booking unpaid	Slot is released after ten minutes and appointment expires.
AT-07	Submit demo payment	Status and reference are stored; card/CVV fields do not exist.
AT-08	Record blood sugar above 140	A HIGH alert is created with the non-diagnostic disclaimer.
AT-09	Publish diet plan and add progress	Plan becomes visible and progress sorts by date.
AT-10	Simulate notification failure	FAILED is saved and changes to simulated delivered on retry.
AT-11	Submit two-star feedback	Feedback and one urgent open complaint are committed together.
AT-12	Request summary with dates	Counts and average rating match database fixtures.
AT-13	Complete full patient journey	Registration through reporting succeeds without manual database edits.
AT-14	Use UI at 320 px and keyboard only	Navigation/forms remain usable with visible focus.

Evidence record

For every run, record date, build identifier, tester, environment, test ID, input fixture, actual result, pass/fail, screenshot or response evidence, and linked defect. Never capture a real password or sensitive record in evidence.

Exit criteria

All high-priority acceptance tests pass, no critical/high defect is open, migrations work on an empty database, backend tests and frontend production build succeed, and the demo script completes twice using a freshly seeded environment.

User Guide

Sign in

Use a demo account from the main README and the demo password. The navigation changes according to role. Accounts lock after five failed attempts; ask the system administrator to unlock a demo account.

Patient

Register, choose a practitioner and available slot, confirm the demo invoice, then use the dashboard to review appointments, results and plans. Mark meals complete, add water intake, send only non-urgent questions, and submit feedback after a completed consultation.

Doctor or medical coordinator

Open the assigned patient, review earlier results and record the current measurements. The system compares them with demonstration thresholds. Complete consultation notes and create a follow-up recommendation when appropriate.

Dietitian

Review the health-alert queue, compare current and previous measurements, create a plan with targets/exclusions/meals, publish it and monitor adherence. Use messages for non-urgent guidance.

Reception and finance

Reception can create, move and cancel bookings while viewing the central schedule. Finance can inspect invoices and record a safe demo payment status. Never type a real card number or CVV.

Administrator and management

Administrators manage roles and locked accounts and review audit activity. Managers filter operational and satisfaction reports and assign complaint follow-up. Exported figures represent synthetic demo data.

Scrum Plan and Requirements Traceability

Eight-week plan

Sprint	Weeks	Goal	Demonstrable outcome
1	1-2	Foundation and access	Register, login, RBAC, shared UI and schema contracts
2	3-4	Appointments and billing	Conflict-safe booking through demo invoice/payment
3	5-6	Health, diet and progress	Check-up alert through published plan and progress log
4	7-8	Communication and insight	Reminders, messages, feedback, complaints, reports and release

Traceability

Requirement	Use case	API group	Acceptance test	Owner
FR-01-04	UC-01	/auth, /users, /patients	AT-01-04	IT25101803
FR-05-08	UC-02	/schedules, /appointments, /invoices, /payments	AT-05-07	IT25103681
FR-09-10	UC-03	/checkups, /health-alerts	AT-08	IT25102636
FR-11-12	UC-04	/diet-plans, /progress-logs	AT-09	IT25101696
FR-13-14	UC-05	/messages, /notifications	AT-10	IT25103601
FR-15-18	UC-06	/feedback, /complaints, /reports, /audit-events	AT-11-13	IT25100792

Definition of done

Code compiles, owner tests pass, validation and failure behavior are covered, migrations run from a blank database, OpenAPI/docs are updated, no prohibited data is introduced, and the integrated journey remains operational.

Final Demonstration Script

1. Sign in as the administrator, show role separation and unlock a prepared locked account.
2. Register a fictional patient and show the protected patient dashboard.
3. As reception/patient, select a practitioner slot and create a ten-minute hold.
4. Record a simulated payment status and show the generated invoice reference.
5. As doctor, record blood sugar of 145 mg/dL and show the high-priority non-diagnostic alert.
6. As dietitian, review the change, publish a low-sugar plan and show the patient notification.
7. As patient, complete a meal, add water and show the weight trend.
8. Exchange a non-urgent care message and show a simulated reminder/retry entry.
9. Submit a two-star review and show the automatically created urgent complaint.
10. As operations manager, open the monthly summary and explain visits, satisfaction and payment totals.

Keep the demo under twelve minutes. Use only seeded fictional records. If an external-looking action is shown, explicitly state that payment, email and SMS are simulated.

Git and Contribution Workflow

Branches

- `main` must always build.
- Each member works in `feature/-`.
- Pull requests target `main`; direct pushes are avoided.

Pull-request checklist

- Change stays inside the owner module unless an integration change is declared.
- New/changed endpoint is reflected in `openapi.yaml`.
- Database change uses a new Flyway version and safe fictional fixtures.
- Relevant tests pass and evidence is linked.
- Authorization and prohibited-data rules were checked.
- README/module docs describe setup or behavior changes.
- At least one other member reviews cross-module changes.

Commit style

Use `feat:`, `fix:`, `test:`, `docs:`, `refactor:` or `chore:` followed by a concise description. Each module's `docs/CONTRIBUTION.md` records task, commits/PRs, review partner, tests and evidence location.

Defect Log

ID	Date	Reporter	Module	Severity	Reproduction	Expected	Actual	Owner	Status	Evidence
-	-	-	-	-	-	-	-	-	No defects recorded	-

Severity: Critical blocks the demo or exposes data; High breaks a core workflow; Medium has a workaround; Low is cosmetic or documentation-only.